

BÀI TẬP
Bài số 2
(Đề thi gồm có **01** trang)

Question 1 (2.0 points). Please read the following code snippet:

```
1 #include <climits>
2 #include <iostream>
3 using namespace std;
4
5 bool check(int *Arr, int l,int r) {
6     while (l < r) {
7         if (Arr[l] >= Arr[l + 1]) return false;
8         l++;
9     }
10    return true;
11 }
12
13 int Get(int *Arr,int l,int r) {
14     int sum = 0;
15     for (int i = l; i <= r; i++)
16         sum = sum + Arr[i];
17     return sum;
18 }
19
20 int main() {
21     int *Arr;
22     int n;
23
24     cin >> n;
25     Arr = new int[n];
26     for (int i = 0; i < n; i++) cin >> Arr[i];
27     int res = -INT_MIN;
28
29     for (int i = 0; i < n; i++)
30         for (int j = i; j < n; j++)
31             if (check(Arr,i,j) == 0) {
32                 break;
33             }
34             else {
35                 int sum = Get(Arr,i,j);
36                 if (res < sum) res = sum;
37             }
38
39     cout << res;
40     return 0;
41 }
```

Input từ bàn phím

8
-3 -1 2 5 3 6 -2 5

- What is the output of the program?
- Please explain why the program produces this output.

Question 2 (2.0 points). Given an array **arr** consisting of n elements indexed from 0 to $n - 1$. An array is called a "beautiful array" if there exists a position x such that: the elements from index 0 to x are sorted in increasing order, and the elements from index x to $n - 1$ are sorted in decreasing order. In other words, $a_i > a_{i-1}$ ($0 < i \leq x$) and $a_i < a_{i-1}$ ($x < i \leq n - 1$). Note that x can be 0 or $n - 1$.

- Implement the function **bool CheckArr(int *arr, int n)** that checks whether the array **arr** is a beautiful array.
- Implement the function **int MaxSubArr(int *arr, int n, int k)** that returns the length of the longest contiguous subarray in **arr** such that the subarray is a beautiful array and the difference between the largest and smallest numbers in the subarray does not exceed k .

Example: Given $n = 5, k = 3$, and $arr = \{2, 4, 8, 10, 7, 2\}$

- The function **CheckArr(int *arr, int n)** returns **true**.
- The function **int MaxSubArr(int *arr, int n, int k)** returns **3**, corresponding to the subarray $\{8, 10, 7\}$.

Question 3 (2.0 points). A Programming Fundamentals class has N students, and each student has M test scores. The score of the j^{th} test ($0 \leq j < M$) for the i^{th} student ($0 \leq i < N$) is denoted as $P_{i,j}$. Note that the students are indexed from 0, and the tests are indexed from 0.

- Implement the function **void findMaxScore(int **P, int n, int m)**. You need to print the index of the student who has the highest average score across all tests. If there are multiple students with the same highest average score, print all their indices.
- Since the average score across all tests is very important for each student, the school allows each student to drop exactly k tests to maximize their average score.

Implement the function **int** ToMaxScore(int **P, int n, int m, int k)**. The function should return a 2D array where each row corresponds to a student, and the columns contain the scores of the $m - k$ tests that the student keeps, ensuring the highest possible average. The order of the scores in the resulting array must match their original order.

Example: Given $N = 4, M = 5, k = 2$, and the matrix **P**:

2	4	5	2	5
7	7	8	6	4
3	6	4	5	3
5	5	3	7	1

- Output 1 as the index of the student with the highest average score, which corresponds to the scores (7, 7, 8, 6, 4).

b. Return the 2D array after dropping k tests for each student to maximize their average score:

4	5	5
7	7	8
6	5	3
5	5	7

Question 4 (2.0 points). Given a singly linked list where each *node* contains an integer value x . You are provided with a pointer *head* pointing to the first *node* of the list.

a. Write a function to **swap two arbitrary nodes in the linked list**.

- Only the *nodes* should be swapped, and not the values inside them.
- The function should take the pointer *head*, and the positions pos_1 and pos_2 of the two *nodes* to be swapped as input.

b. Given two singly linked lists *head_1* and *head_2*, both already **sorted in ascending order**:

- Write a function to **merge the two lists into a single sorted linked list**.
- After merging, the resulting list should be stored in *head_1*. The pointer *head_2* must be set to `null`.

Input Data

1. For part a:

- Pointer *head*: points to the first node of the linked list.
- Two positions pos_1 and pos_2 : specify the two nodes to be swapped (positions are 1-based).

2. For part b:

- Two linked lists *head_1* and *head_2*, both sorted in ascending order.

Each node in the linked list has the following structure:

```
1 struct Node {  
2     int value;  
3     Node* next;  
4 };
```

Example

Part a:

Initial list:

$1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9$

Swap nodes at positions 2 and 4:

Positions: $pos_1 = 2, pos_2 = 4$

Result after swapping:

$1 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 9$

Part b:

Input lists:

head_1 : 1 → 3 → 5 → 7

head_2 : 2 → 4 → 6 → 8

Result after merging:

head_1 : 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

head_2 : null

Question 5 (2.0 points). The file `INPUT.TXT` contains a list of information about students in a school. Each line contains the following information: **Student ID, Full Name, GPA (Grade Point Average), Year of Birth**. The fields are separated by the character ' - '.

- Declare a structure data type with variables to store the following information: **Student ID, Full Name, GPA**, and **Year of Birth**. Then read the data from `INPUT.TXT` into an array of this structure type.
- Write a function to find all students with the highest GPA and save their information into the file `OUTPUT.TXT`.
- Students in the same department share the first 5 digits of their Student IDs. Write a function to print the department code (first 5 digits) with the most students. If there are two or more departments with the same number of students, select the department with the largest code. Then print the names of all students belonging to that department to the screen.

Example for part b:

INPUT.TXT	OUTPUT.TXT
23125022-Hoang Duy Vu-9.1-2005	23125014-Nguyen Thanh Nhan-9.3-2005
23125014-Nguyen Thanh Nhan-9.3-2005	23127025-Trinh Tran Phuong Tuan-9.3-2005
23127025-Trinh Tran Phuong Tuan-9.3-2005	
23127023-J97-6.3-2005	

Example for part c:

INPUT.TXT	OUTPUT
23125022-Hoang Duy Vu-9.1-2005	23127
23125014-Nguyen Thanh Nhan-9.3-2005	Jack
23127025-Trinh Tran Phuong Tuan-9.3-2005	Nguyen Van X
23127023-Jack-6.3-2005	Trinh Tran Phuong Tuan
23127052-Nguyen Van X-8.3-2005	

——— END ———

Notes:

- Candidates are **not allowed** to use any materials during the exam.
- Invigilators are not allowed to explain anything further.